

Documentação - Pokédex API

Joyce Moura da Silva¹, Roberta Manassés Madeira², Samuel De Alcantara Magalhães³

¹Licenciatura em Computação – Universidade Federal do Paraná (UFPR)
Rua Pioneiro – Palotina – PR – Brazil

1. Visão Geral

A Pokédex API é uma aplicação back-end distribuída que integra uma API REST com um banco de dados NoSQL (MongoDB) e uma API pública (PokéAPI). O projeto demonstra conceitos fundamentais de sistemas distribuídos: comunicação via HTTP, persistência em bancos não-relacionais, consumo de serviços externos, autenticação JWT e testes via Postman.

2. Requisitos Funcionais

Tabela 1. Requisitos Funcionais Implementados

Código	Descrição	Status
RF01	Banco de Dados NoSQL	Implementado (MongoDB)
RF02	API REST com 5 Métodos HTTP	Implementado (GET, POST, PUT, PATCH, DELETE)
RF03	Integração com API Pública Gratuita	Implementado (PokéAPI)
RF04	Documentação das Rotas	Implementado (Swagger + RE-ADME)
RF05	Testabilidade via Postman	Implementado (Collection pronta)

3. Requisitos Não-Funcionais

Tabela 2. Requisitos Não-Funcionais Implementados

Código	Descrição	Status
RNF01	Estrutura do Projeto	Organizado em pastas (routes, controllers, models e services)
RNF02	Variáveis de Ambiente	Arquivo .env com credenciais
RNF03	Tratamento de Erros	Respostas HTTP com códigos corretos
RNF04	Dados Válidos	Validação mínima dos campos
RNF05	README	Instruções de execução do projeto
RNF06	Containerização Docker	Docker e Docker Compose

4. Fluxo de Dados

Fluxos Principais da API Pokémon

1. Buscar Pokémon (GET /api/pokemons)

Cliente → API → MongoDB → JSON Response

2. Importar Pokémon (POST /api/pokemons)

Cliente → API → PokéAPI → MongoDB → JSON Response

3. Atualizar Pokémon (PUT/PATCH /api/pokemons/:id)

Cliente → API (validar JWT) → MongoDB → JSON Response

4. Deletar Pokémon (DELETE /api/pokemons/:id)

Cliente → API (validar JWT) → MongoDB → JSON Response

5. Login (POST /api/auth/login)

Cliente → API → Gera JWT Token → JSON Response

4.1. Stacks Utilizadas

Stack Tecnológicas

Backend

- Node.js – Runtime JavaScript
- Express.js – Framework Web
- MongoDB – Banco de Dados NoSQL
- Mongoose – ODM (Object Document Mapper)
- JWT (jsonwebtoken) – Autenticação
- Axios – Cliente HTTP para integração com APIs
- Swagger/OpenAPI – Documentação interativa
- Docker – Containerização
- Postman – Testes

Frontend

- React – Biblioteca para construção da interface
- Axios – Cliente HTTP

API Externa

- PokéAPI – Dados dos Pokémons
- Disponível em: <https://pokeapi.co/api/v2>

Ferramentas de Desenvolvimento

- Nodemon – Reinicialização automática do servidor
- Git – Controle de versão
- Docker Compose – Containers

4.2. Guia de Instalação e Execução

```
# 1. Clonar repositório
git clone https://github.com/joyceemoura/pokedex-api.git
cd pokedex-api

# 2. Configurar variáveis de ambiente
cp .env.example .env

# 3. Subir com Docker
docker compose up --build

# Para rodar o Frontend
git clone https://github.com/joycemoura/pokedex-frontend
cd pokedex-frontend

npm install (para instalar as dependências)
npm start (inicializar)
```

4.3. Acessar a aplicação

```
API:      http://localhost:3000
Health:   http://localhost:3000/health
Swagger:  http://localhost:3000/api-docs
Frontend: http://localhost:4000 (após npm start)
```

5. Estrutura do Projeto

```
pokedex-api/
  src/
    config/
      database.js      # Conexão MongoDB
      swagger.js       # Configuração Swagger
      auth.js          # Configuração JWT
    models/
      Pokemon.js       # Schema de Pokémon
      Captura.js       # Schema de Histórico
    controllers/
      pokemonsController.js # Lógica de Pokémon
      pokedexController.js  # Lógica de Pokédex
    routes/
      auth.js          # Rotas de autenticação
      pokemons.js     # Rotas de pokémons
      pokedex.js       # Rotas de pokédex
```

```

    services/
      pokeApiService.js      # Integração com PokéAPI
    app.js                   # Arquivo principal Express
  pokdex-frontend/
    src/
      App.js                 # Componente principal React
      App.css                # Estilos
    public/
      package.json
      README.md
  .env.example               # Template de variáveis
  .env                      # Variáveis locais
  .gitignore                # Arquivos ignorados
  .dockerignore             # Arquivos ignorados
  Dockerfile                # Imagem da API
  docker-compose.yml        # Orquestração Docker
  package.json              # Dependências Node
  README.md                 # Documentação rápida
  postman_collection.json    # Collection Postman

```

6. Configurar Bearer Token

No Postman, para acessar as rotas protegidas:

1. Clique na aba **Authorization**.
2. Selecione a opção **Bearer Token**.
3. Cole o token recebido do login.
4. Pronto! Todas as requisições protegidas funcionam.

7. Documentação das Rotas da API

7.1. Rotas de Pokémons

POST /api/auth/login

i Descrição: Autentica o usuário e retorna um JWT token. (Credenciais: *admin / 123456*)

Autenticação:  **Nenhuma**

✓ Status HTTP: 200 OK, 401 Unauthorized

GET /api/pokemons/

Descrição: Retorna a lista completa de pokémons cadastrados no banco de dados.

Autenticação:  Nenhuma

Status HTTP: 200 OK

GET /api/pokemons/{{pokemon_id}}

Descrição: Retorna os dados completos de um pokémon específico.

Autenticação:  Nenhuma

Status HTTP: 200 OK, 404 Not Found

POST /api/pokemons/charmander

Descrição: Busca um pokémon na PokéAPI pelo nome e o salva no banco local.

Autenticação:  Obrigatória (Bearer Token)

Status HTTP: 201 Created, 400 Bad Request, 401 Unauthorized, 409 Conflict

PUT /api/pokemons/{{pokemon_id}}

Descrição: Substitui TODOS os campos de um pokémon existente.

Autenticação:  Obrigatória (Bearer Token)

Status HTTP: 200 OK, 401 Unauthorized, 404 Not Found

PATCH /api/pokemons/{{pokemon_id}}

Descrição: Atualiza APENAS os campos enviados no corpo da requisição.

Autenticação:  Obrigatória (Bearer Token)

Status HTTP: 200 OK, 401 Unauthorized, 404 Not Found

PATCH /api/pokemons/{pokemon_id}/capturar

Descrição: Altera o status de um pokémon, marcando-o como capturado.

Autenticação:  Obrigatória (Bearer Token)

Status HTTP: 200 OK, 401 Unauthorized, 404 Not Found

PATCH /api/pokemons/{pokemon_id}/favorito

Descrição: Marca ou desmarca um pokémon específico como favorito (Toggle).

Autenticação:  Obrigatória (Bearer Token)

Status HTTP: 200 OK, 401 Unauthorized, 404 Not Found

DELETE /api/pokemons/{pokemon_id}

Descrição: Remove permanentemente um pokémon do banco de dados.

Autenticação:  Obrigatória (Bearer Token)

Status HTTP: 200 OK, 401 Unauthorized, 404 Not Found

7.2. Rotas da Pokédex

GET /api/pokedex/resumo

Descrição: Retorna as estatísticas gerais e contadores da sua pokédex.

Autenticação:  Nenhuma

Status HTTP: 200 OK

GET /api/pokedex/capturados

Descrição: Lista exclusivamente os pokémons que foram marcados como capturados.

Autenticação:  Nenhuma

Status HTTP: 200 OK

GET /api/pokedex/favoritos

Descrição: Lista exclusivamente os pokémons que foram marcados como favoritos.

Autenticação:  Nenhuma

Status HTTP: 200 OK

GET /api/pokedex/historico

Descrição: Retorna o registro do histórico de buscas e importações realizadas na API.

Autenticação:  Nenhuma

Status HTTP: 200 OK